Enhanced Entity relationship

Enhanced Entity relationship diagram (EERD) :

- Inheritance + ERD : inheritance (is a relationship)
- Done when making the ERD , if we found that we can make inheritance between types.
- So before making the relations and mapping , make the base type and other types inherit from it.
- It's not a must to implement the whole ERD as EERD , we can mix between them.
- It's better to work with EERD , but if It's not applicable and we cannot find inheritance relationships then make an ERD.

Before EERD , we had strong entity and weak entity only.

Now we have Supertype entity and Subtype entity also.

- Supertype : A generic entity type that has a relationship with one or more subtypes. Supertype entity Shared attributes and relationships between the subtypes (Generalization).
- Subtype : A subgrouping of the entities in an entity type which has attributes that are distinct from those in other subgroupings. Subtype entity has the Specific attributes and relationships for this subtype (specification).

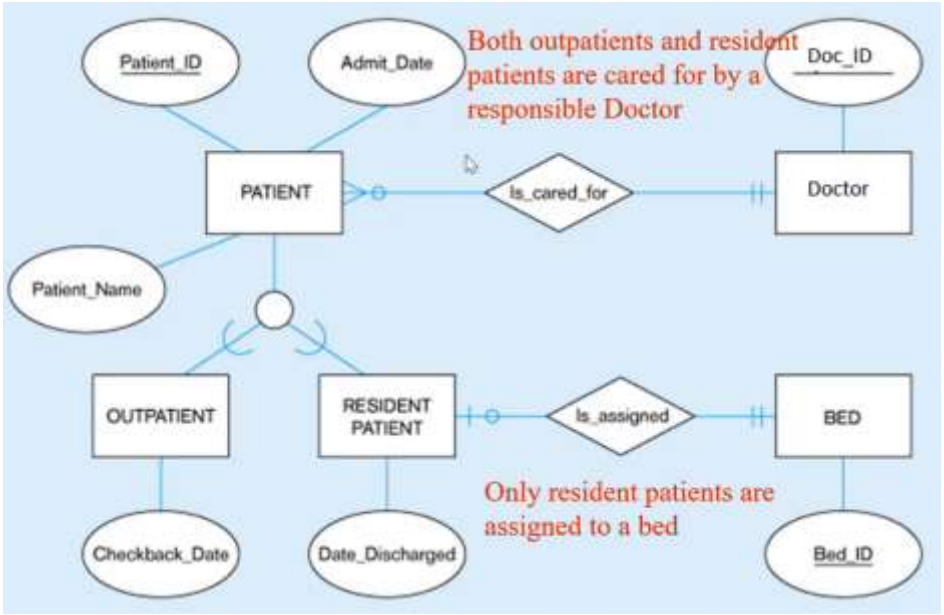
Inheritance :

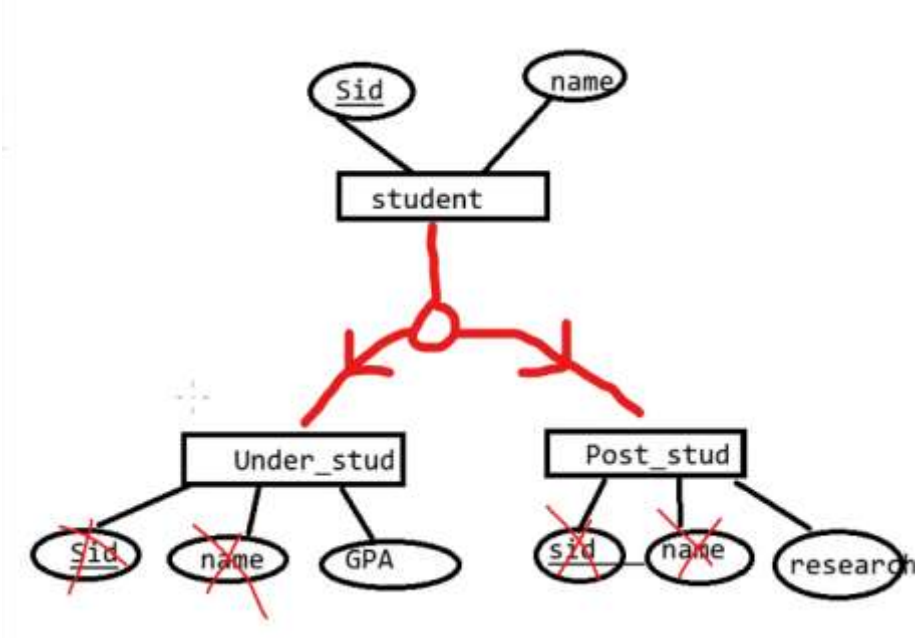
- Subtype entities inherit values of all attributes of the supertype.
- An instance of the subtype is also an instance of the supertype.

Relationships:

- Relationships at the supertype level indicate that all subtypes will participate in the relationship.
- The instance of a subtype may participate in a relationship unique to that subtype. In this situation, the relationship is shown at the subtype level.

Ex: All patients(outpatient , resident patient) are in relationship with doctor entity, but resident patient has a unique relationship with bed entity.

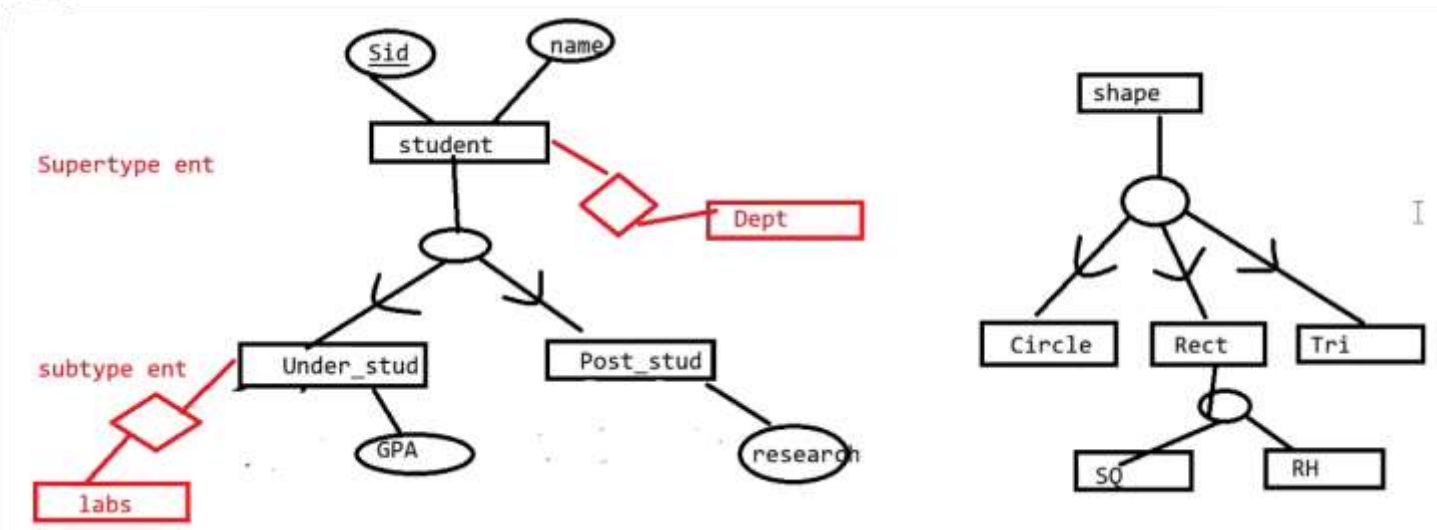


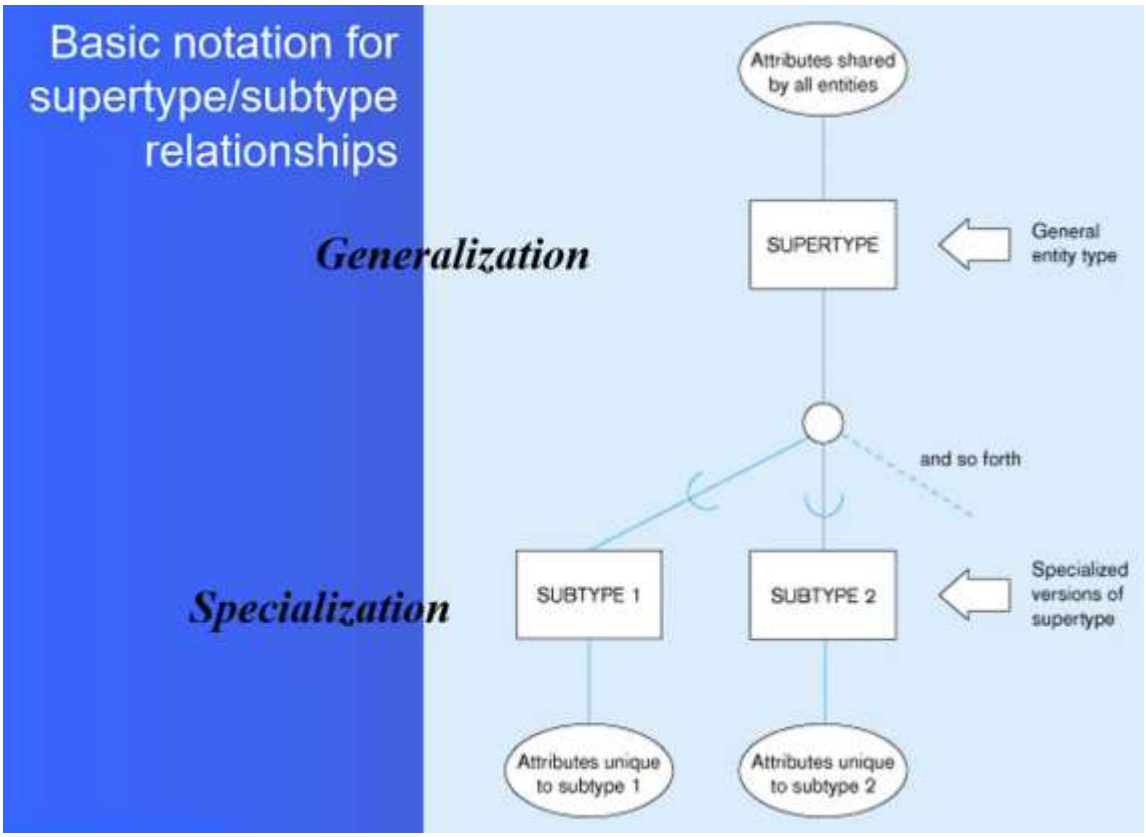


EERD helps us :

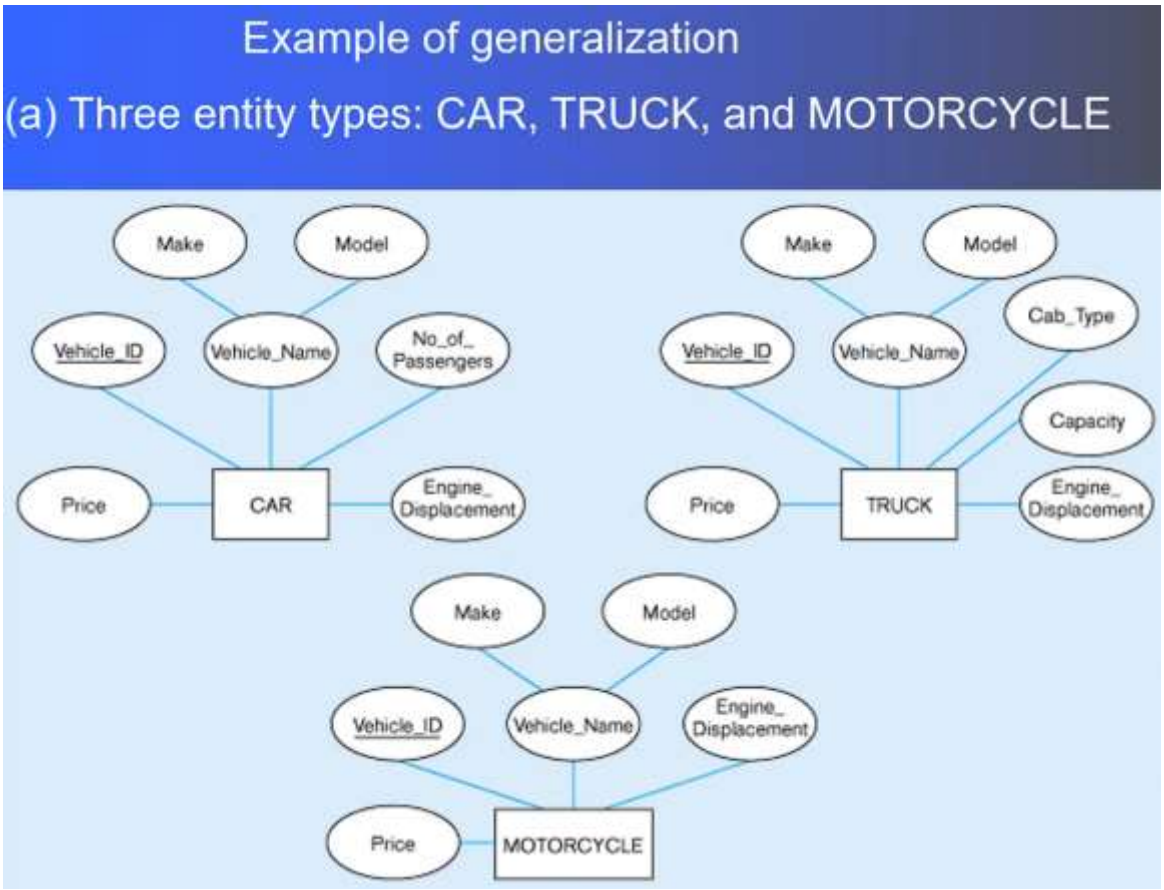
- Understanding the business and the system in a better way.
- Scaling , as in the future we can add more Subtypes.
- Helps us in implementation of the application and classes.
- If the subtypes are in relationship with other entity , then it's enough to make the relationship between the supertype and the other entity.

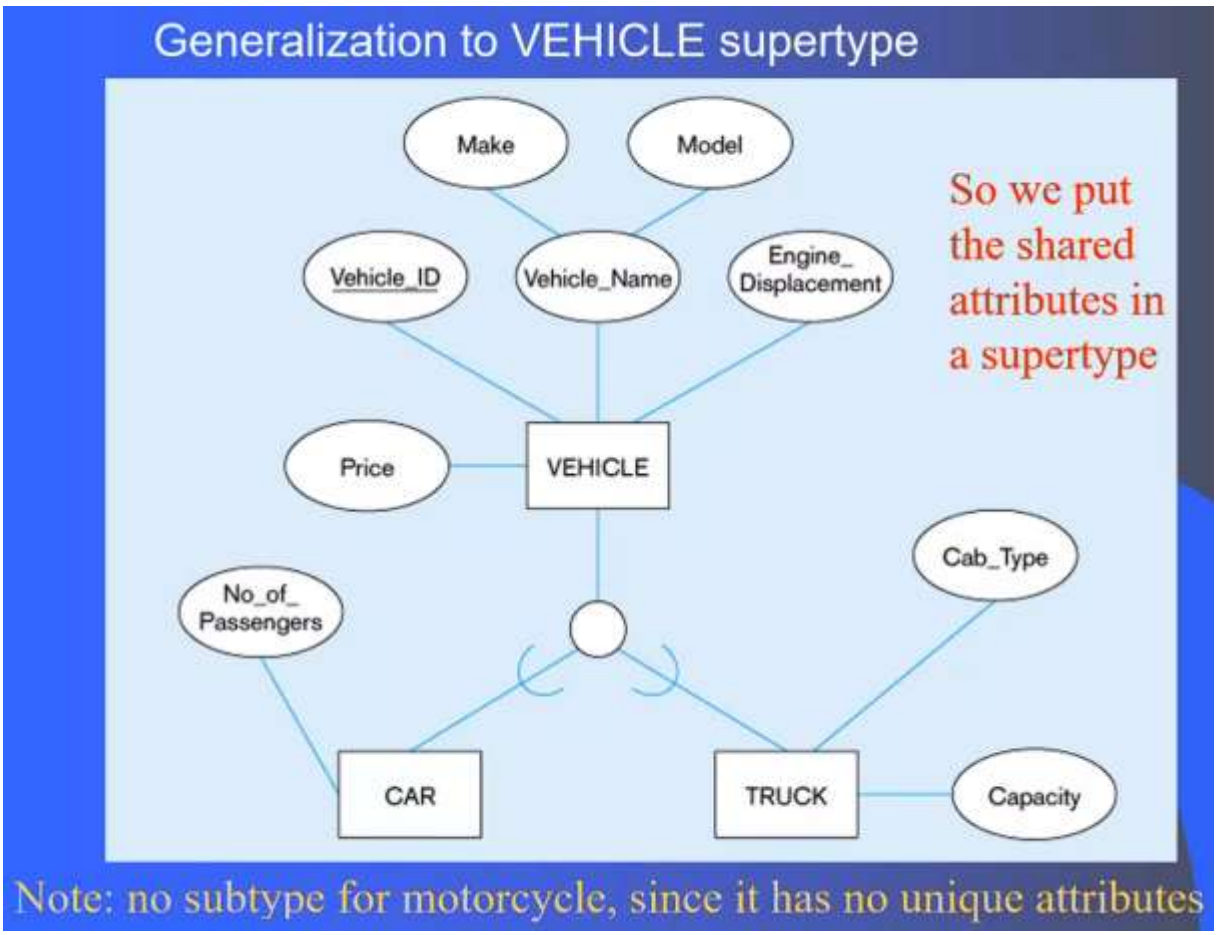
Note : We can make multi-level inheritance , ex: (shape => rectangle => square)




Basic notation for supertype/subtype relationships
Generalization
Specialization
Attributes shared by all entities
SUPERTYPE
General entity type
and so forth
SUBTYPE 1
SUBTYPE 2
Specialized versions of supertype
Attributes unique to subtype 1
Attributes unique to subtype 2

Example 1 :

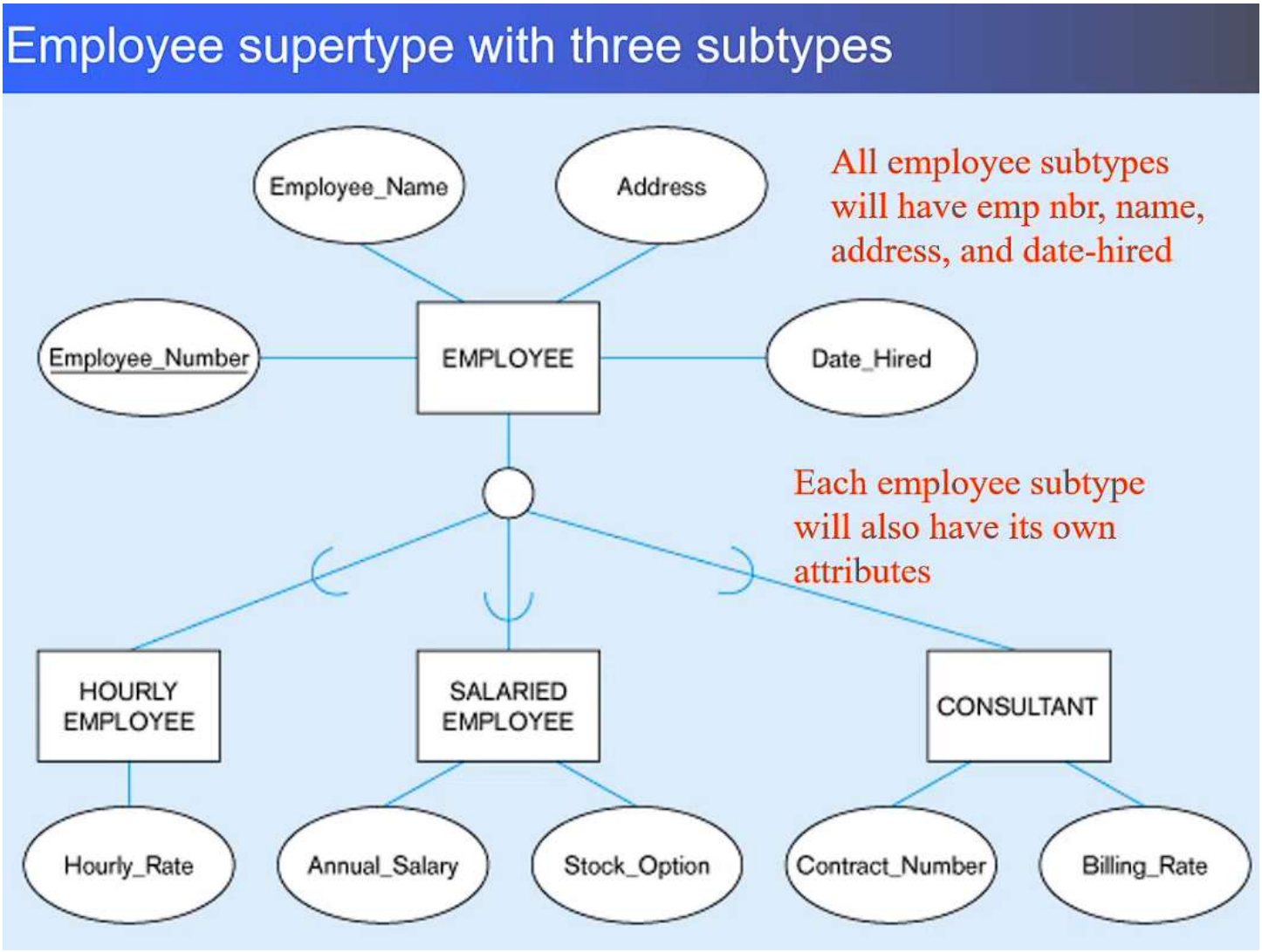

Example of generalization
(a) Three entity types: CAR, TRUCK, and MOTORCYCLE
Make
Model
Vehicle_ID
Vehicle_Name
No_of_ Passengers
Price
CAR
Engine_ Displacement
Make
Model
Cab_Type
Vehicle_ID
Vehicle_Name
Capacity
Price
TRUCK
Engine_ Displacement
Make
Model
Vehicle_ID
Vehicle_Name
Engine_ Displacement
Price
MOTORCYCLE



Note : the previous EERD has a problem , because now if there is a relationship between another entity and the motorcycle , then we cannot put it (missing motorcycle subtype and we cannot put it in the supertype). In this case it will be better if we found an attribute that is unique for the motorcycle only, and then make a subtype for motorcycle to participate in the relationship.

Note : An entity without attributes is NOT an entity , even it it has relationships between other entities but to be an entity it must have at least one attribute.

Example 2 :

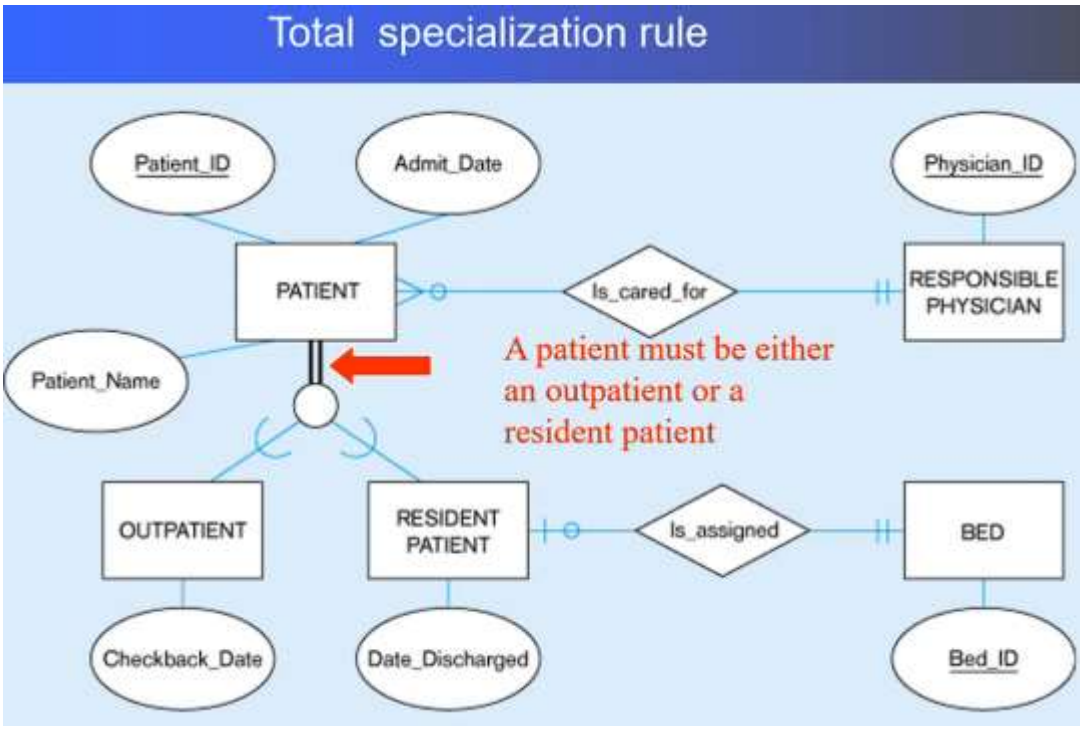


Constraints in Supertype :

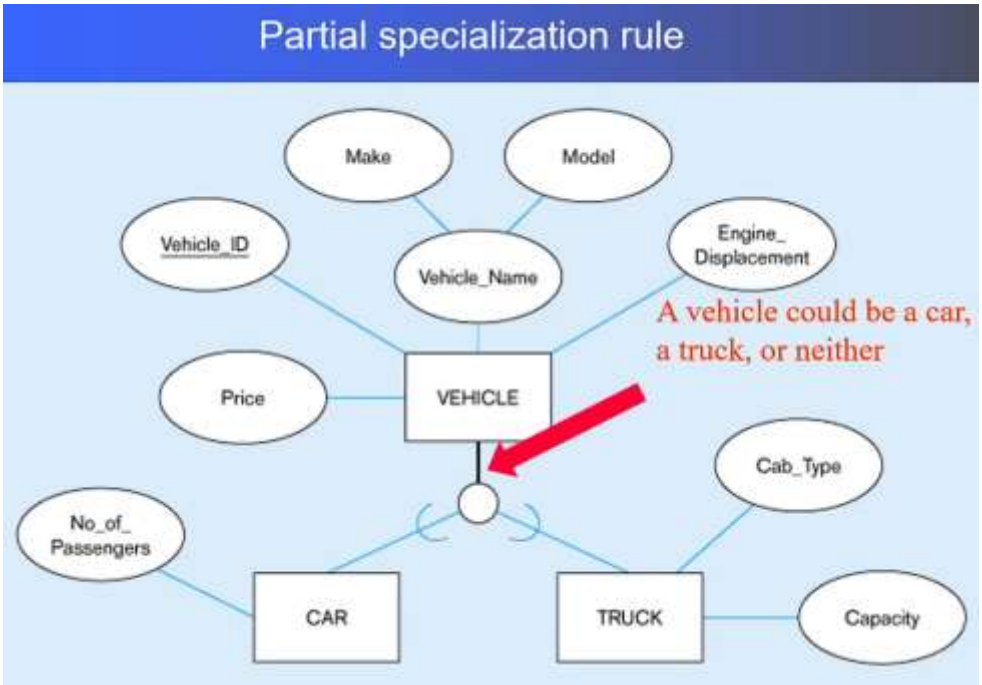
- Completeness Constraints:
  - Total Specialization Rule (double line)
  - Partial Specialization Rule (single line)
- Disjoint Constraints :
  - Total Disjoint (d)
  - Overlap rule (o)

Completeness Constraints :

- Total Specialization Rule : double line from the parent to subtypes , means that EVERY PARENT is categorized as one of the childs , we cannot find a parent that is not a type of child.

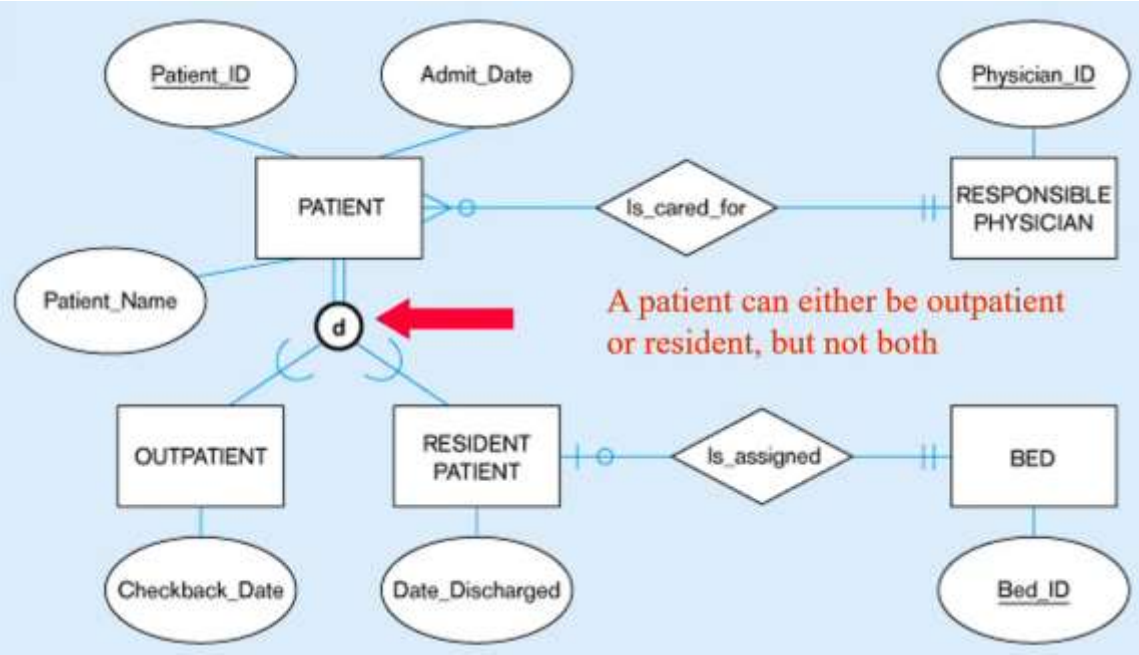


- Partial Specialization Rule : single line from the parent to subtypes , means that NOT every parent is categorized as one of the childs , we can find parents that are not in categorized as any subtype. (ex: a vehicle can be a motorcycle)

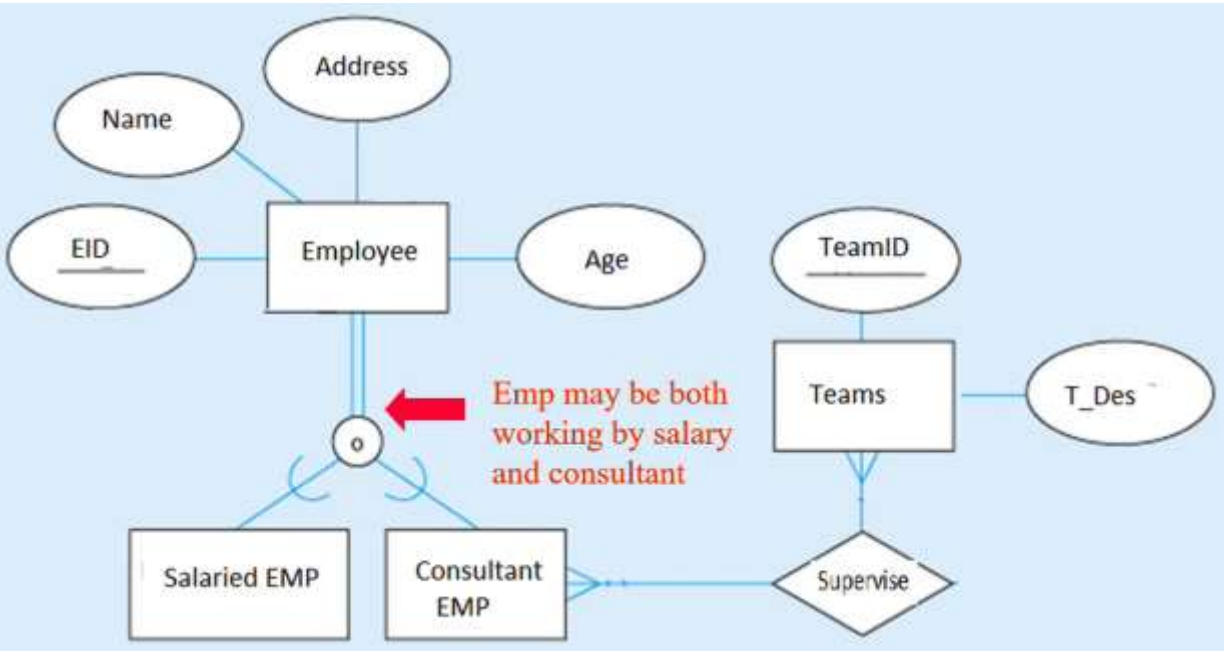


Disjoint Constraints :

- Total Disjoint (d) : one parent CANNOT be categorized as multiple subtypes or childs. Ex: Patient can be either outpatient or resident patient (not both).

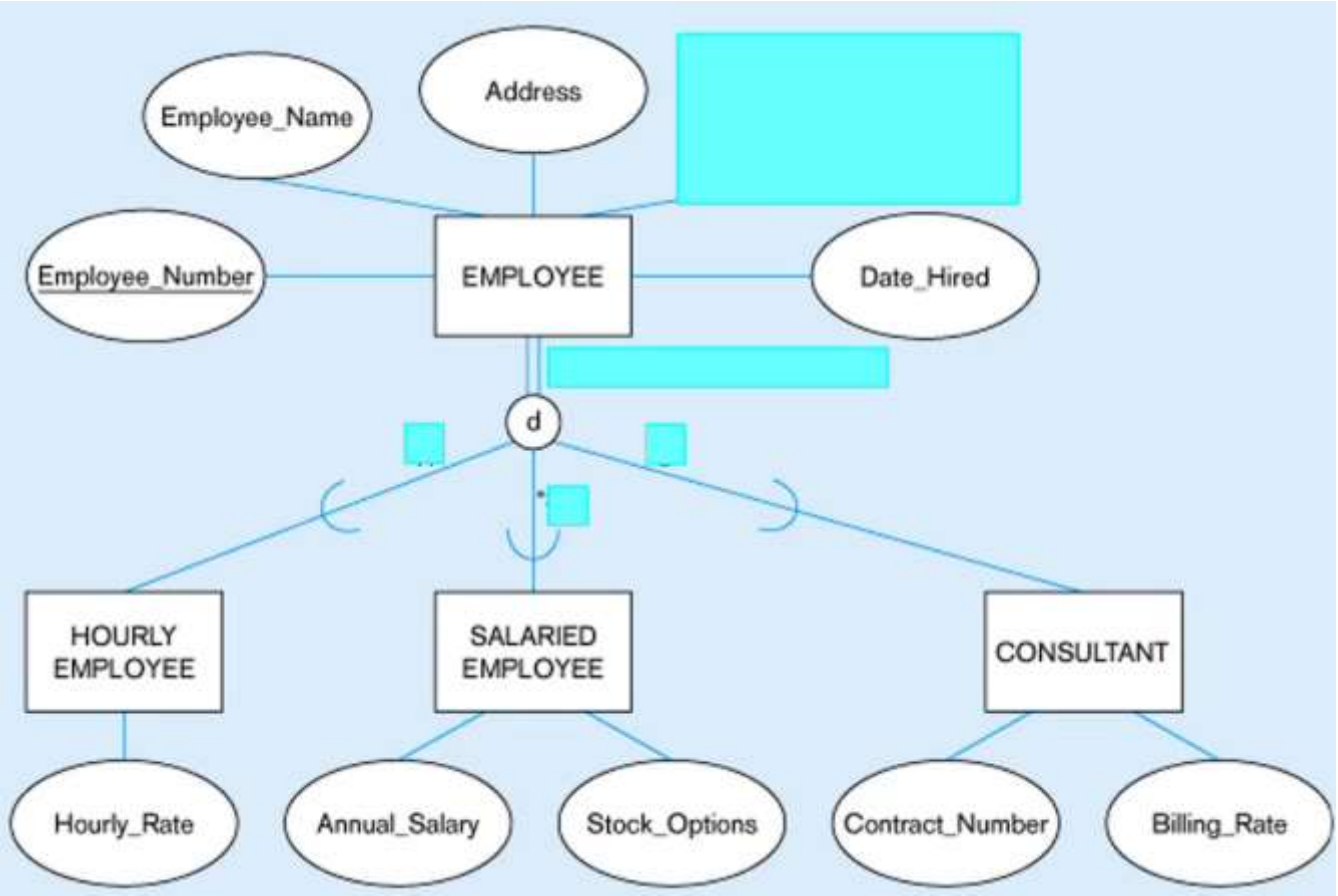


- Overlap rule (o) : one parent can be categorized as multiple subtypes or childs. Ex: Employee can be a salaried employee and a consultant , both in the same time.



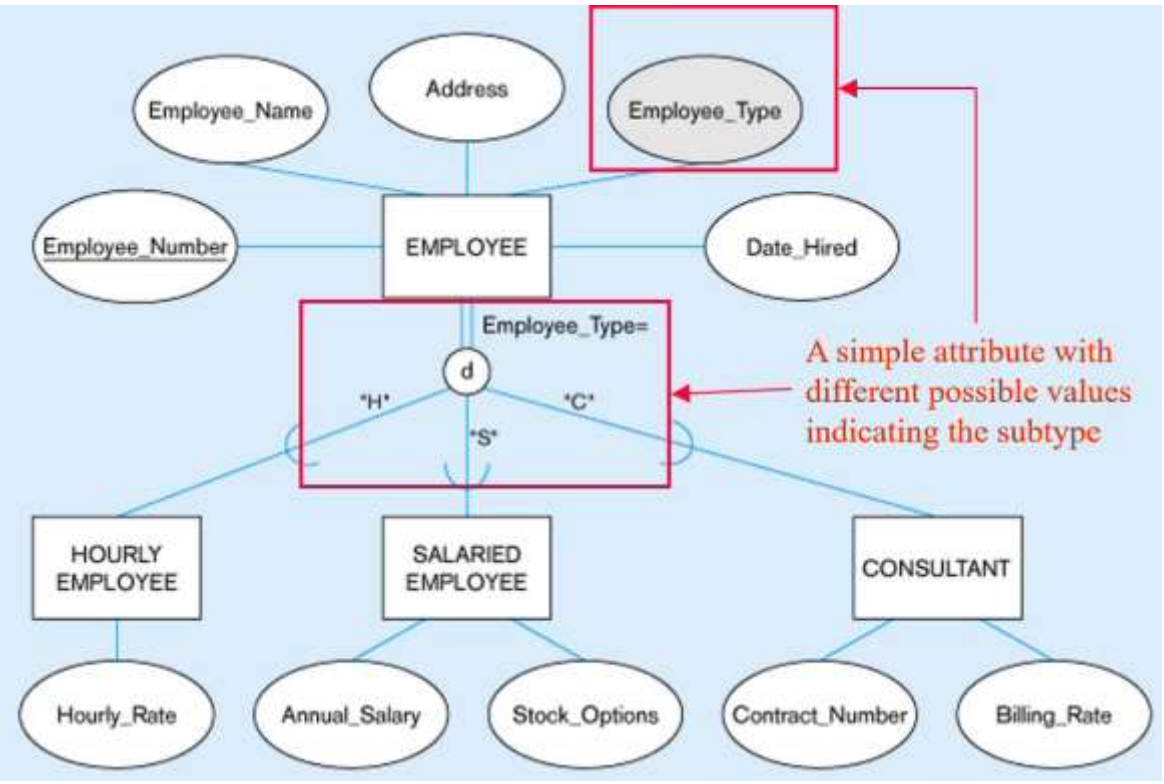
What is a subtype discriminator ?

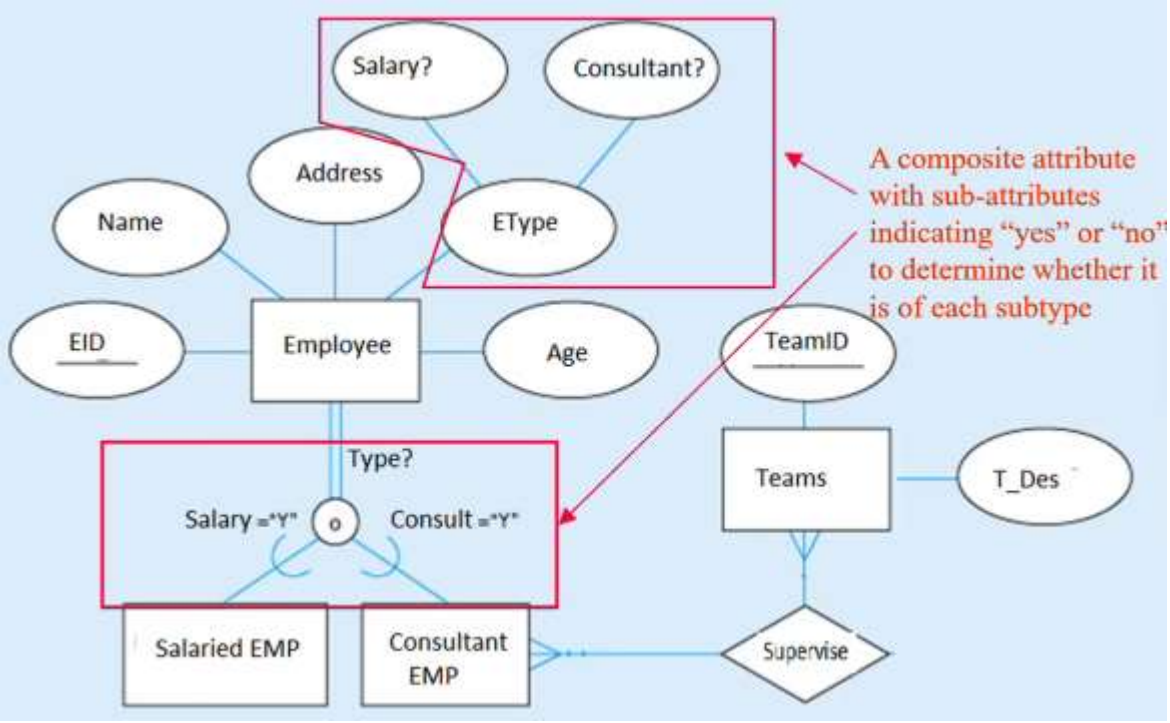
- Next example => without using discriminator :



- Get names of Employees => select from ONE table (Employee table).
- Get names of employees and their annual salary => select from TWO tables (join Employee and Salaried EMP)
- Get names of employees and their annual salary OR Contact number => select from THREE tables (join Employee and Salaried EMP and Consultant)
- Get the name of employee who takes salary (without showing the salary) => select from TWO tables (join Employee and Salaried EMP) , even if we don't want the salary but we joined the two tables !! (Bad , solved by discriminator)
- Get the name of employee who takes salary or has Contact number (without showing the salary or the Contact number) => select from THREE tables (join Employee and Salaried EMP and Consultant), even if we don't want the salary or Contact number but we joined the three tables !! (Bad , solved by discriminator)

Using Discriminator : we will add an attribute to the parent or supertype to specify it's subtype (discriminator), usually takes values (int or char). This discriminator can be composite type as in (Disjoint Constraints Overlap rule) , this may have disadvantages due to having many subtypes, with many subtypes we can make if multivalued instead of composite or work totally without discriminator !!



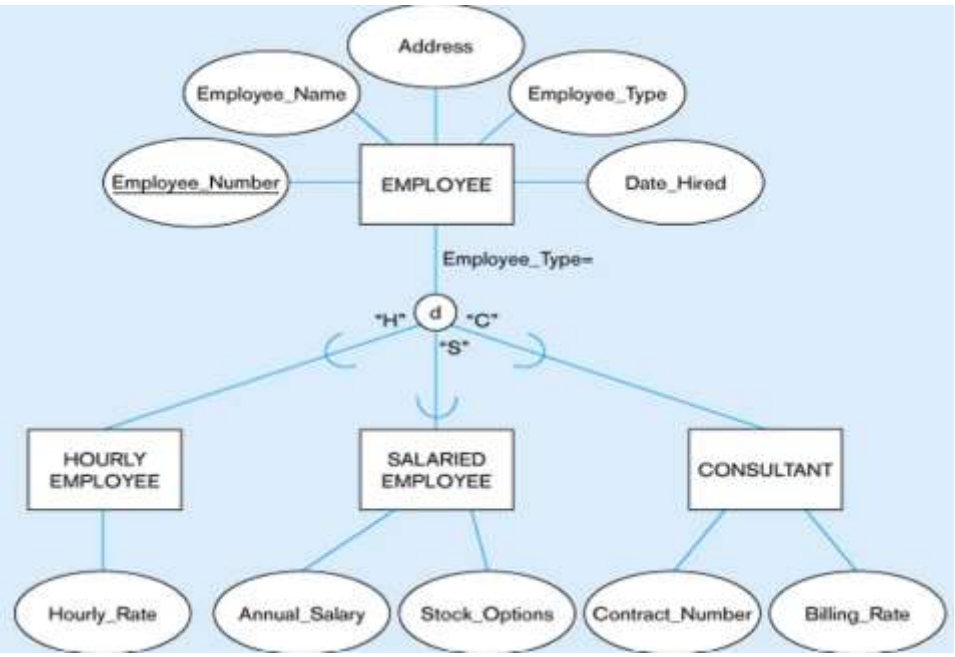


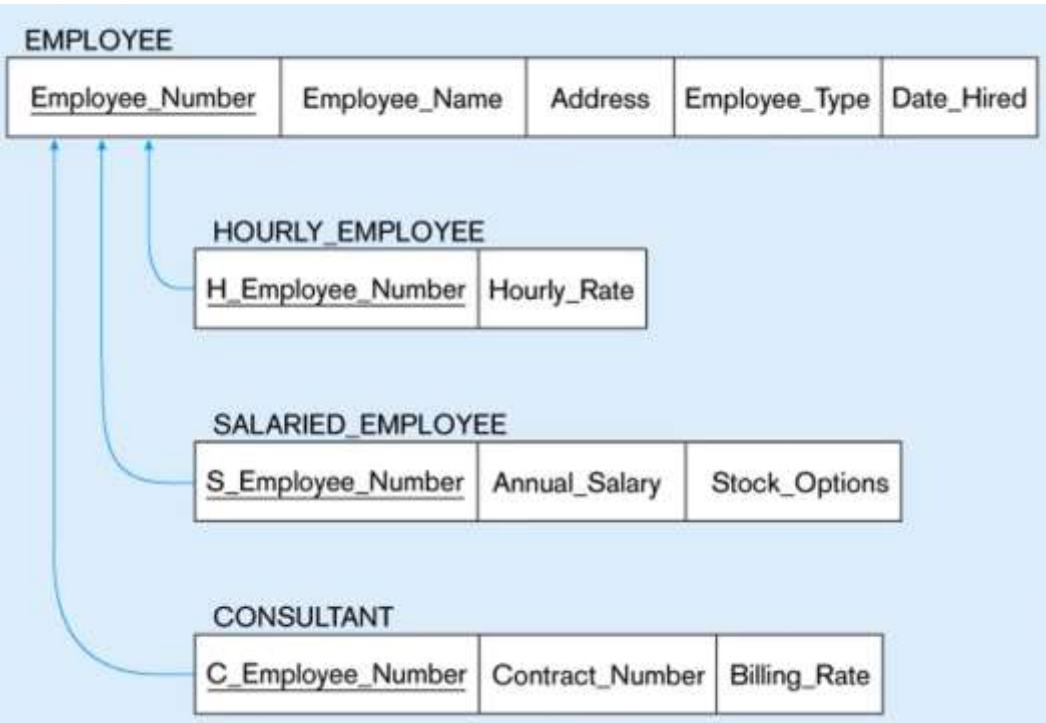
Now we know the type without joining , we join only when wanting data from the other table.

- Get the name of employee who takes salary (without showing the salary) => select from ONE tables (where Employee_Type = 'S')

How to map EERD ?

- Each entity is a table, supertype is a table , and also each subtype is a table, all of them having the same PK , but the PK in subtypes is a PK and also a FK.
- When inserting data in the tables , we must first insert in the supertype and then insert in the subtypes.





Important examples :

- Total specialization (two lines) + disjoint (d) => ONE NUMBER
- Partial specialization (two lines) + disjoint (d) => RANGE (X TO Y)
- Total specialization (two lines) + Overlap (o) => RANGE (X TO Y)
- Partial specialization (two lines) + Overlap (o) => RANGE (X TO Y)